

Documentation

VITYARTHI PROJECT PYTHON

Seat Manager

Overview

- 1. Introduction**
- 2. Problem Statement**
- 3. Algorithm and flowchart**
- 4. Code and Implementation**
- 5. Discussion and Results**
- 6. References**

1. Introduction

Seat Manager

- This document provides a comprehensive overview of the Seat Manager, from its problem statement and design to the implementation details and results.

2. Problem Statement

2.1 Definition

- The main issue is the need for a simple, ongoing way to manage seat occupancy in small places like a library, a self-study room, or a computer lab. The system must track who is using each seat, show which seats are available, and update seat statuses in real-time.
- All data must be saved so that the application retains its state when the program is closed and reopened.
- This project is part of the Vityarthi Python Essentials course. It is a command-line application that meets specific needs. It also acts as a hands-on exercise in basic Python programming, data management, and application structure.

2.2 Requirements

- The functional requirements for the system are as follows:
- **User Management:**
 - i. Create new users with a name, email, and a unique system-generated ID.
 - ii. View a list of all registered users.
- **Seat Management:**
 - i. Display a list of all available seats and their current status (i.e., "Empty" or occupied by a specific user).
 - ii. Allocate an empty seat to a registered user.
 - iii. Release an occupied seat, making it available for others.

- **Data Persistence:**
 - i. Load the existing user and seat data from local files (`.json`) when starting the application.
 - ii. Save all changes to users and seats back to the local files when exiting the application.
 - iii. Provide an option to exit without saving any changes.
- **Interface:**
 - i. Users interact with the system through a simple, text-based menu in the command line.
 - ii. The project is intentionally limited in scope. It does not include user authentication, operates on a small scale for 10-50 seats, and uses basic error handling.
- The project is limited in scope. It does not include user authentication and operates on a small scale, designed for 10-50 seats. It also uses basic error handling.

3. Algorithm and Flowchart

3.1 Algorithm

- The application's logic follows a straightforward algorithm:

i. Initialization:

- At startup, the application reads data from ``data/users.json`` and ``data/seats.json``.
- If these files are missing, it starts with a default set of empty seats and an empty user list.
- This data is loaded into manager objects in memory (``UserManager`` and ``SeatManager``) that hold the application's current state.

ii. Main Event Loop:

- The application enters an infinite loop to continuously serve the user.
- Inside the loop, a menu of options appears for the user.
- The program waits for the user to make a choice.

iii. Process User Input:

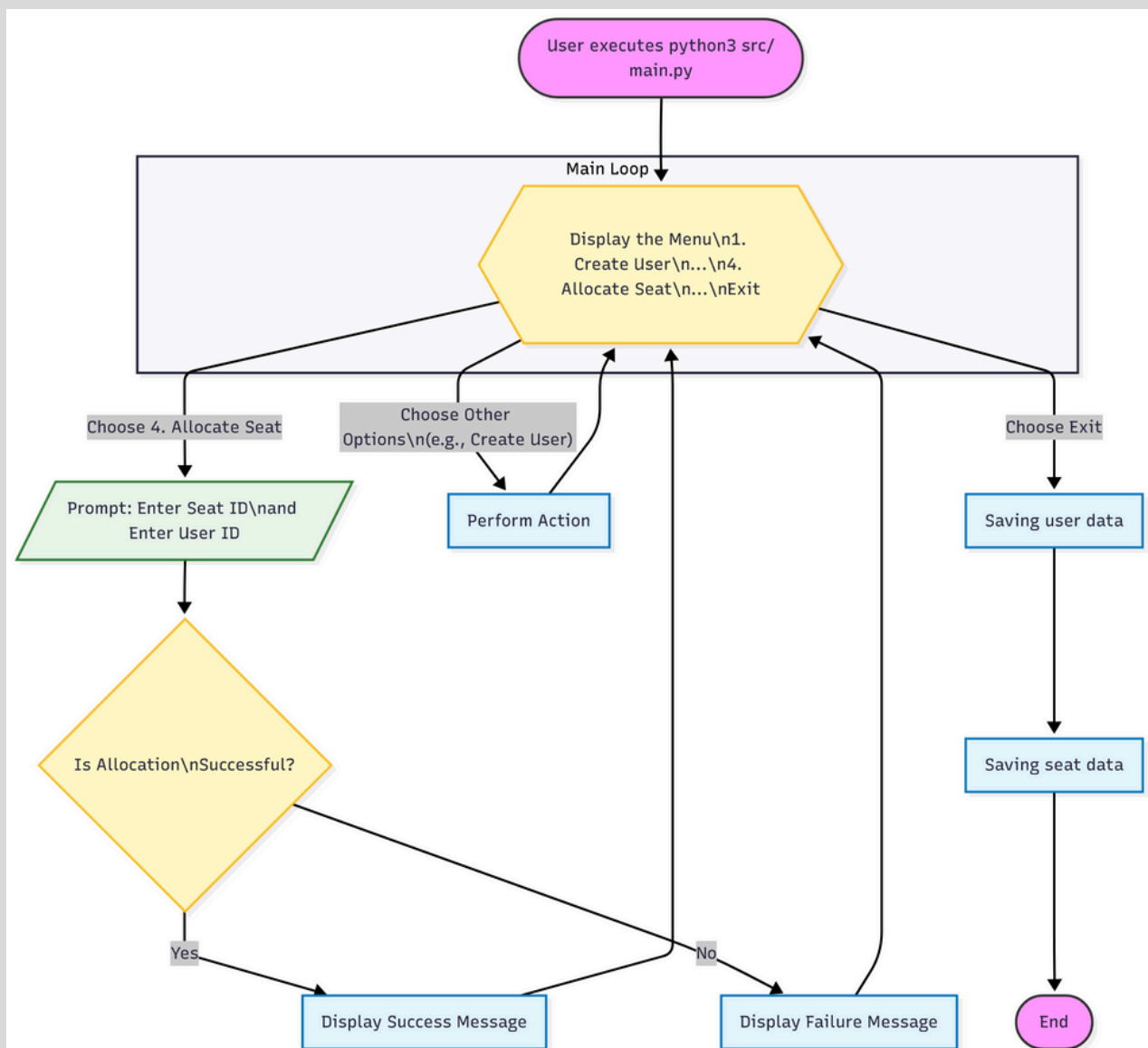
- Based on the user's choice, the program performs a specific action:
- Create User: Prompts for a name and email, checks the email, generates a unique ID, and adds the new user to the ``UserManager``.
- List Users/Seats: Retrieves the current data from the respective manager and displays it in a formatted way on the console.
- Allocate Seat: Prompts for a seat ID and a user ID. It checks that the seat exists, is empty, and that the user ID is valid before assigning the seat.

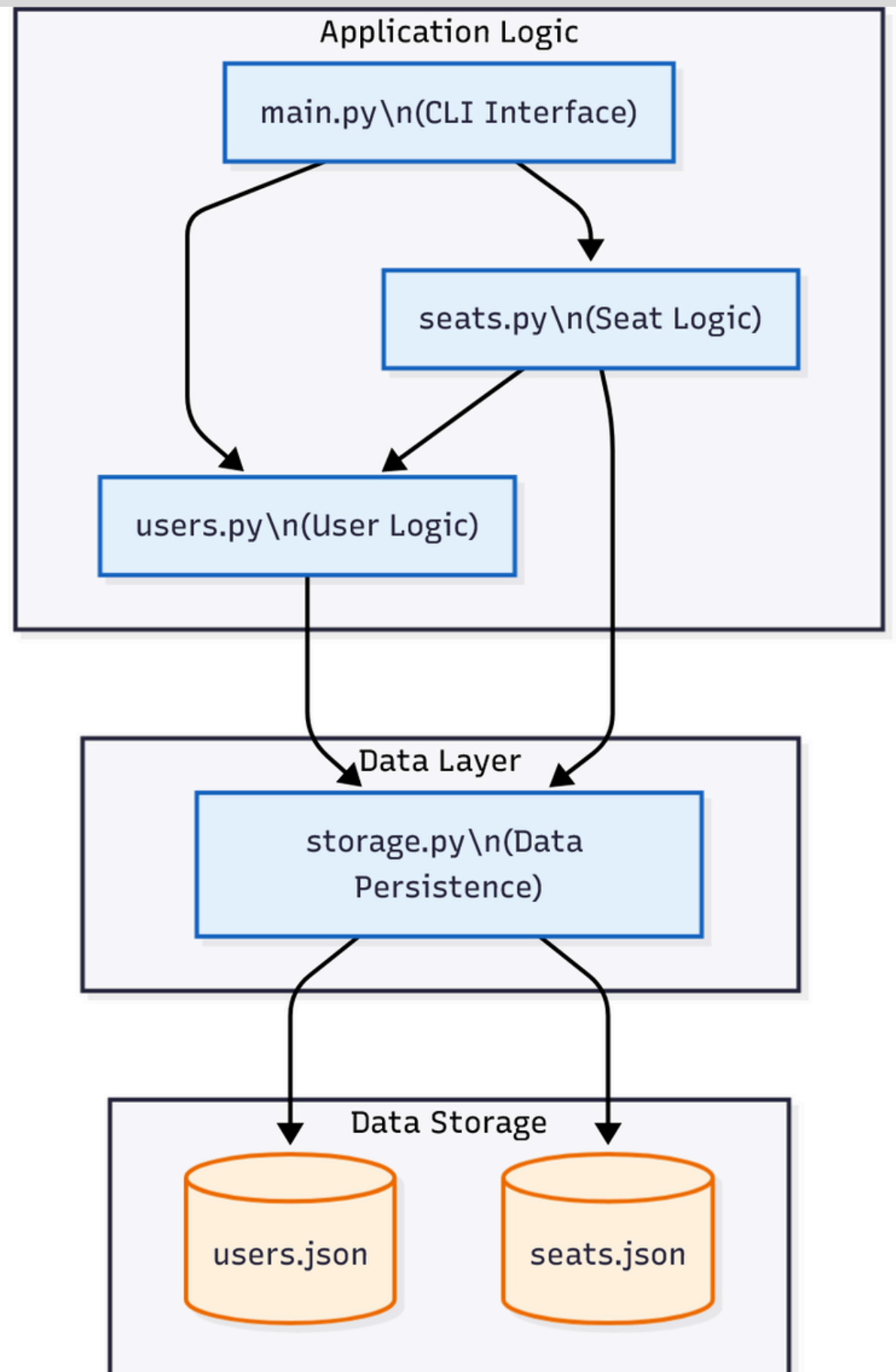
- **Release Seat:** Prompts for a seat ID and checks that the seat exists and is currently occupied before marking it as empty.
- **Save & Exit:** The `UserManager` and `SeatManager` objects are changed into a format suitable for JSON. The program writes the current state back to `users.json` and `seats.json`, then exits the loop.
- **Exit Without Saving:** The program immediately exits the loop, discarding all changes made during the session.

iv. Termination:

- The program exits when the user chooses either exit option.

3.2 Flowcharts





4. Code and Implementation

The project consists of four main Python modules in the `src/` directory. This setup ensures a clear separation of responsibilities.

i. `storage.py` - The Data Layer

This module handles all interactions with the filesystem.

- `load_users(path)/ load_seats(path)`:
These functions read the JSON files. They include error handling for cases where a file is missing, returning sensible default data like an empty user list or a default set of 10 seats, or if the file content is corrupted.
- `save_users(path, data)/ save_seats(path, data)`:
These functions take the in-memory data from the managers and write it to the respective JSON.

files, using an indent of 2 for easy readability.

ii. `users.py` - The User Logic

This module defines the `UserManager` class, which includes all business logic related to users.

- `__init__(self, initial_data)`: The constructor sets up the manager with a list of users loaded from the `storage` module.
- `create_user(self, name, email)`:
This method creates a new user. It validates the email address to ensure it contains "@" and "." characters.
- It also generates a unique 8-character ID using Python's `uuid` library and adds the new user dictionary to its internal list.
- `list_users(self)`:** This method returns the complete list of users.
- `find_user(self, user_id)`:** This method searches through the user list to find and return a user dictionary that matches the given `user\_id`.

iii. `seats.py` - The Seat Logic

This module defines the `SeatManager` class for all operations related to seats.

- `__init__(self, initial_data)`: The constructor initializes the seat dictionary with the data loaded from the `storage` module. It converts the JSON string keys back into integer keys for the seat dictionary.
- `allocate(self, seat_id, user_id, user_mgr)`: This is the main allocation method. It performs several checks:
 1. Does the `seat\_id` exist?
 2. Is the seat already occupied (`is not None`)?
 3. Does the `user\_id` correspond to a real user by calling `user\_mgr.find\_user()`?If all checks pass, it assigns the `user\_id` to the seat.
- `release(self, seat_id)`: This method releases a seat by setting its value back to `None` after confirming that it exists and is currently occupied.
- `to_dict(self)`: This helper method prepares the in-memory seat data for saving. It converts the integer keys of the `self.seats` dictionary back to strings to ensure compatibility with the JSON format.

iv. `main.py` - The Controller

This is the main entry point of the application, connecting all the other modules.

- * It imports the manager classes and storage functions.
- * The `main()` function manages the entire process. It calls the `load\_\*` functions, creates instances of the managers, and contains the main `while` loop that operates the user interface.
- * It gathers input from the user and calls the right methods on the `UserManager` and `SeatManager` instances based on that input.

5. Discussion and Results

- The application meets all the requirements in the problem statement. When you run ``python src/main.py``, it shows a clear and functional command-line menu.

i. **User Creation and Listing:** Users can be created easily. The system correctly lists all users along with their IDs, names, and emails.

ii. **Seat Allocation:**

The system accurately displays the status of all seats. The allocation feature works as intended, linking a user to a seat. The validation logic properly prevents assigning an already-occupied seat or a seat to a non-existent user.

iii. **Data Persistence:**

The "Save & Exit" option saves the state of both users and seats in the ``data/`` directory. When you restart the application, this state is successfully loaded, showing that the data persistence layer works well. The "Exit Without Saving" option also functions correctly by discarding any changes made during the session.

- The project shows a solid grasp of Python basics and software design principles like separation of concerns. The division of logic between presentation (``main.py``), business rules (``users.py``, ``seats.py``), and data persistence (``storage.py``) makes the code neat, readable, and easy to maintain.
- The screenshots in the ``readme.md`` file, including ``menu_usercreation.png`` and ``List_and_allocation_seats.png``, visually confirm the described functionality.
- While the project is successful, a significant issue found during the review is a bug in the test suite: the file ``tests/test_users.py`` is an exact copy of ``tests/test_seats.py`` and does not contain any actual tests for the ``UserManager`` class. This is a common issue in early-stage projects and indicates an area for future improvement.

6. References:

- This project was created from scratch for a course.
- It does not use any external frameworks or libraries, except for standard Python 3.14.
- The main sources of information for understanding the project are its source code and documentation.
- **Project Source Code:**
Located in the “src/” directory.
- **Project Statement:**
‘statement.md’
- **README File:**
‘readme.md’

Thank You